



MODULE MIA4 · JOUR 2 / 5

Modélisation, Indexation & Drivers

Jour 2 — Schema design, index et connexion applicative

Concevoir un schéma MongoDB adapté · Créer et optimiser les index · Analyser les plans d'exécution · Se connecter depuis Python, Java, PHP

Plan de la journée

7 heures effectives · théorie, ateliers guidés, TP libre

- 1 **Rappel Jour 1 & questions** — 9h00-9h30
- 2 **Schema design MongoDB** — embed vs reference · 9h30-10h30
- 3 **Patterns de modélisation** — bucket, subset, computed · 10h45-12h00
-  *Pause déjeuner — 12h00-14h00*
- 4 **Indexation & profiling** — explain plan · 14h00-15h30
- 5 **Transactions ACID multi-doc** — 15h30-16h00
- 6 **Drivers Python / Java / PHP** — atelier PyMongo · 16h00-17h00

HORAIRES

9h00 → 17h00

- 🕒 7h effectives
- ☕ Pause 10h30-10h45
- 🍴 Déjeuner 12h-14h
- 🖥️ 2 TP guidés

MATÉRIEL REQUIS

- ✓ Instance MongoDB Jour 1
- ✓ Compass installé
- ✓ Python + pip + venv
- ✓ pymongo 4.x
- ✓ IDE (VS Code / PyCharm)

Rappel Jour 1 — transition vers la modélisation

🕒 Points clés du Jour 1

- ✓ **Big Data — 5V** : Volume, Vitesse, Variété, Véracité, Valeur.
- ✓ **SQL vs NoSQL** : le choix dépend du besoin, pas d'une "supériorité".
- ✓ **CAP** : Consistency, Availability, Partition — 2 sur 3.
- ✓ **4 familles NoSQL** : clé-valeur, document, colonnes, graphe.
- ✓ **MongoDB** = document, BSON, scale horizontal, CP.
- ✓ **Installation** Docker/local · client `mongosh` et Compass.
- ✓ **CRUD** : insertOne/Many · find · updateOne/Many · deleteOne/Many.

📈 Ce que nous ajoutons aujourd'hui

Nous savons stocker des documents — mais **comment les structurer** pour être performants et évolutifs ?

Question du jour

Faut-il **tout imbriquer** dans un seul document ou **référencer** comme en SQL ?

- » Décision embed vs reference
- » Patterns de modélisation éprouvés
- » Créer les bons index — et éviter les mauvais
- » Analyser un plan d'exécution
- » Connecter une app Python à MongoDB

Schema design MongoDB — la question centrale

En SQL on normalise (3NF). En MongoDB on modélise autour des **accès applicatifs**.

📌 Le principe fondateur

Le modèle NoSQL est piloté par les **requêtes de l'application**, pas par une théorie de normalisation.

Règle d'or : *"Data that is accessed together should be stored together"* — Rick Houlihan, AWS re:Invent 2018.

On accepte la **redondance contrôlée** pour minimiser les jointures et maximiser la performance de lecture.



1 Identifier les entités

Lister les concepts métier (utilisateur, commande, produit) et leurs attributs. C'est proche de la démarche SQL classique.



2 Cartographier les requêtes

Lister **toutes** les requêtes que fera l'application : lectures fréquentes, écritures fréquentes, agrégations analytiques. La forme du schéma en découle.



3 Choisir embed vs reference

Décider pour chaque relation : **imbriquer** le sous-document dans le parent, ou **référer** par ObjectId comme une clé étrangère.

Embed (imbriquer) vs Reference (lier)

Embed — sous-documents imbriqués

```
{ "_id": ObjectId("..."), "client": "Alice", "lignes": [ { "produit": "Clavier", "prix": 49.9, "qte": 1 }, { "produit": "Souris", "prix": 19.9, "qte": 2 } ] }
```

Choisir Embed quand...

- ✓ La relation est **1:1** ou **1:peu** ($\leq \sim 100$)
- ✓ Les enfants n'ont pas d'existence propre
- ✓ Les données sont **toujours lues ensemble**
- ✓ La taille totale reste bien < 16 Mo
- ✗ Éviter si la sous-doc grossit sans limite

Reference — ObjectId vers autre collection

```
// commande { "_id": ObjectId("cmd1"), "client_id": ObjectId("cli_alice"), "lignes": [ObjectId("prd_kbd"), ObjectId("prd_mouse")] } // résolution via $lookup ou 2e requête
```

Choisir Reference quand...

- ✓ La relation est **1:beaucoup** ou **n:m**
- ✓ L'entité liée existe indépendamment
- ✓ Les données sont **rarement lues ensemble**
- ✓ Les données changent souvent (éviter la duplication)
- ✗ Coût : une requête / \$lookup supplémentaire

Patterns de modélisation MongoDB

6 patterns récurrents documentés par MongoDB Inc. pour résoudre des problèmes classiques.



Bucket Pattern

Regrouper N mesures temporelles (IoT, logs) par tranche horaire dans un même document. Réduit dramatiquement le nombre de docs et améliore les scans.



Subset Pattern

N'embarquer que les N premiers éléments d'une liste illimitée (ex : 10 derniers commentaires) dans le document, garder le reste ailleurs.



Computed Pattern

Précalculer et stocker les résultats d'agrégation coûteuse (moyennes, totaux). Idéal pour dashboards lus 100× plus qu'écrits.



Polymorphic Pattern

Stocker plusieurs types d'entités similaires (voiture, moto, camion) dans une seule collection avec un champ discriminant type.



Schema Versioning

Ajouter un champ `schema_version` — permet la coexistence de versions et les migrations progressives.



Extended Reference

Dénormaliser une **petite partie** des données référencées (nom, ville) pour éviter le \$lookup en lecture ; accepter une incohérence contrôlée.

Indexation MongoDB — les principes

Un index est une structure B-Tree qui accélère les recherches — au prix d'écritures plus lentes et d'un coût mémoire.

⚡ Créer un index

```
// Index simple ascendant db.livres.createIndex({ titre: 1 }) // Index
composé (ordre = important) db.livres.createIndex({ auteur: 1, annee: -1
}) // Index unique db.users.createIndex({ email: 1 }, { unique: true })
```

☰ Lister & supprimer

```
// Voir tous les index d'une collection db.livres.getIndexes() //
Stats d'usage (utile !) db.livres.aggregate([{$indexStats:{}}]) //
Supprimer un index db.livres.dropIndex("titre_1")
```

✅ Avantages

- Lectures **10 à 1000 fois** plus rapides sur les champs indexés
- Support natif des **tri** et **plages** (\$gte, \$lt)
- **Contraintes d'unicité** gérées par MongoDB
- Utilisable par le planificateur pour les jointures (\$lookup)

⚠ Coûts & pièges

- Chaque écriture met à jour tous les index → **ralentit les inserts**
- Occupation **RAM** supplémentaire (un index doit tenir en mémoire)
- **_id** est indexé automatiquement, pas besoin de le refaire
- Un index inutilisé = **coût pur**, pensez à surveiller \$indexStats

Les principaux types d'index MongoDB

Type d'index	Usage	Exemple
Simple	Un seul champ, ascendant ou descendant	<code>createIndex({ nom: 1 })</code>
Composé	Plusieurs champs. Règle ESR : Equality, Sort, Range	<code>createIndex({ status:1, date:-1 })</code>
Multi-clés	Indexe chaque élément d'un tableau — automatique si le champ contient un tableau	<code>createIndex({ tags: 1 })</code>
Text	Recherche full-text — stemming, stop-words	<code>createIndex({ contenu: "text" })</code>
Géospatial 2dsphere	Requêtes de proximité GPS (coordonnées GeoJSON)	<code>createIndex({ loc: "2dsphere" })</code>
Hashed	Pour le sharding — répartition uniforme des shard keys	<code>createIndex({ user_id: "hashed" })</code>
TTL (Time To Live)	Supprime automatiquement les docs après X secondes — sessions, caches	<code>createIndex({ createdAt:1 }, { expireAfterSeconds: 3600 })</code>
Partiel / Sparse	N'indexer que les docs matchant un filtre ou ayant le champ — gain mémoire	<code>{ partialFilterExpression: { actif: true } }</code>

Profiling — détecter les requêtes lentes

Le profiler MongoDB enregistre les opérations dépassant un seuil de latence dans `system.profile`.

🔴 Activer le profiler

```
// Niveau 0 : désactivé (défaut) // Niveau 1 : opérations > slowms (100ms
défaut) // Niveau 2 : tout enregistrer (DEV uniquement)
db.setProfilingLevel(1, { slowms: 50 }) // Consulter l'état
db.getProfilingStatus()
```

🔍 Lire les traces

```
// Requêtes lentes récentes db.system.profile.find().sort({ ts: -1
}) .limit(10) // Requêtes qui font un COLLSCAN
db.system.profile.find({ planSummary: "COLLSCAN" })
```

💡 Champs clés à lire dans `system.profile`

op · type d'opération : **query**, **insert**, **update**, **remove**, **command**

ns · namespace **db.collection** concerné

millis · durée en ms — la métrique la plus importante

planSummary · **IXSCAN** (bon, index utilisé) vs **COLLSCAN** (mauvais, scan complet)

docsExamined vs **nreturned** · ratio idéal ≈ 1

keysExamined · nombre d'entrées d'index parcourues

.explain() — le plan d'exécution

Comparaison des stages

Trois modes: `queryPlanner` (défaut) · `executionStats` · `allPlansExecution`.

```
// Utilisation basique db.livres.find({ auteur: "Camus" })
.explain("executionStats") // Extrait de la sortie : { "winningPlan": {
"stage": "FETCH", "inputStage": { "stage": "IXSCAN", "indexName": "auteur_1"
} }, "executionStats": { "nReturned": 42, "totalDocsExamined": 42, // idéal
! "executionTimeMillis": 2 } }
```

✓ IXSCAN — Index Scan

L'index est utilisé. Rapide, coût logarithmique. **C'est ce que vous voulez.**

⚠ FETCH — Récupération

Après IXSCAN, aller chercher le document complet sur disque. Évitable via **projection couverte**.

✗ COLLSCAN — Collection Scan

Aucun index utilisable. Parcours complet de la collection. **Le tuer : créez un index.**

🎯 Métriques clés à surveiller

- `nReturned` — documents renvoyés
- `totalDocsExamined` — documents lus (idéal = `nReturned`)
- `totalKeysExamined` — clés d'index parcourues
- `executionTimeMillis` — temps total en ms

Transactions ACID multi-documents

Depuis MongoDB 4.0 (Replica Set) et 4.2 (Sharded Cluster) — les transactions ACID multi-documents sont disponibles.

🔑 Rappel — ACID vs BASE

Atomicity : tout ou rien — **Consistency** : contraintes respectées — **Isolation** : concurrences sans interférence — **Durability** : engagement persistant.

Sur un **document unique**, MongoDB est ACID **nativement** depuis toujours. Les transactions étendent cela à plusieurs documents / collections / bases.

```
// Exemple mongosh - transfert bancaire atomiqueconst session = db.getMongo().startSession()
session.startTransaction() try { const comptes = session .getDatabase("banque").comptes
comptes.updateOne( { iban: "FR76-A" }, { $inc: { solde: -100 } } ) comptes.updateOne( { iban:
"FR76-B" }, { $inc: { solde: +100 } } ) session.commitTransaction() } catch (e) {
session.abortTransaction() throw e } finally { session.endSession() }
```

💡 Bonnes pratiques

- **Courtes** — cible < 60 sec (limite par défaut)
- **Peu de documents** touchés à la fois
- **Nécessite un Replica Set** (même minimal)
- Coût en performance : ~10-40% vs write simple
- **Retry automatique** côté driver sur
TransientTransactionError
- Envisager d'abord un **bon schéma design** (embed) qui rend
la transaction inutile

Drivers MongoDB — panorama

Drivers officiels maintenus par MongoDB Inc. — API cohérente, sémantique presque identique d'un langage à l'autre.



Python — PyMongo

Le driver le plus populaire. API BSON native, support asyncio via `motor`.

```
pip install pymongo
```



Java —

mongodb-driver-sync

Driver Java 8+, versions sync et reactive-streams disponibles.

Maven / Gradle



PHP —

mongodb/mongodb

Extension `mongodb` + lib PHP officielle via Composer.

```
composer require mongodb/mongodb
```



Node.js — mongodb

100% Promise/async-await. Souvent utilisé via ODM Mongoose pour Express/NestJS.

```
npm install mongodb
```

💡 Concepts communs à tous les drivers

MongoClient — objet racine, thread-safe, à créer **une seule fois** par application (pool de connexions interne)

Database & Collection — objets légers, à obtenir depuis le client à la volée

Connection String URI — `mongodb://user:pass@host:27017/db?authSource=admin&replicaSet=rs0`

CRUD API — `insert_one/many` · `find` · `update_one/many` · `delete_one/many`

BSON types — chaque driver mappe les types BSON aux types natifs du langage (ObjectId, ISODate, Decimal128...)

PyMongo — CRUD depuis Python

```
# bibliotheque.py – CRUD complet avec PyMongo 4.x
from pymongo import MongoClient, ASCENDING, DESCENDING
from pymongo.errors import DuplicateKeyError
from datetime import datetime # 1) Connexion – MongoClient créé UNE fois par application (pool interne)
client = MongoClient("mongodb://admin:ipssi2025@localhost:27017/?authSource=admin")
db = client["bibliotheque"]
livres = db["livres"] # 2) Index (idempotent – pas d'erreur si déjà présent)
livres.create_index([("titre", ASCENDING)], unique=True)
livres.create_index([("auteur", ASCENDING), ("annee", DESCENDING)])
# 3) CREATE
try:
    result = livres.insert_one({ "titre": "L'Étranger", "auteur": "Camus", "annee": 1942, "note": 9.1, "cree_le": datetime.utcnow() })
    print(f"Inserté avec _id={result.inserted_id}")
except DuplicateKeyError:
    print("Titre déjà présent – index unique")
# 4) READ – filtre + projection + tri + limite
for livre in livres.find( {"annee": {"$gte": 2000}}, {"titre": 1, "note": 1, "_id": 0} ).sort("note", DESCENDING).limit(5):
    print(livre) # 5) UPDATE – plusieurs opérateurs
livres.update_many({"auteur": "Camus"}, {"$set": {"favori": True}, "$inc": {"vues": 1}}) # 6) DELETE
n = livres.delete_many({"note": {"$lt": 5}}).deleted_count
print(f"{n} livre(s) supprimé(s)")
client.close()
```

ATELIER 1H30

TP — Modéliser un système d'e-commerce & indexer

☰ Consignes

- 1 **Modéliser** une base ecommerce avec collections `clients`, `produits`, `commandes`. Justifier **embed vs reference** par écrit.
- 2 **Générer** 1 000 clients, 200 produits et 5 000 commandes avec `Faker` (Python).
- 3 **Écrire 5 requêtes analytiques** — top 10 clients, panier moyen, produits en rupture, CA mensuel, clients inactifs +6 mois.
- 4 **Exécuter** `.explain("executionStats")` — noter **COLLSCAN** vs **IXSCAN** et les temps.
- 5 **Créer les index nécessaires** et re-mesurer. Créer un tableau *avant / après*.
- 6 **Coder un mini-script PyMongo** qui affiche le top 5 clients par CA.
- 7 **Bonus** : ajouter un index **text** sur la description produit et tester `$text`.

🚩 LIVRABLES

- `modele.md` · schéma + justifications
- `seed.py` · génération données
- `queries.js` · 5 requêtes
- `bench.md` · comparatif index
- `top5.py` · script analytique

💡 ÉVALUATION

- Pertinence des choix embed/ref
- Qualité du script de seed (idempotent)
- Bon usage des opérateurs `$aggregate`
- Analyse critique du `explain`
- Bonus : requête couverte (covered query)

Ce qu'il faut retenir du Jour 2



Modéliser par requêtes

Le schéma suit les accès de l'appli, pas la 3NF.
Data accessed together, stored together.



Embed pour 1:peu, Ref pour 1:beaucoup

Règle simple : cardinalité, taille bornée, cycle de vie commun ?



6 patterns utiles

Bucket, Subset, Computed, Polymorphic, Versioning, Extended Reference.



Index : ESR, mesuré

Equality → Sort → Range. Un index inutilisé est un coût — mesurez avec \$indexStats.



explain() est vital

IXSCAN = bon, COLLSCAN = à corriger. Ratio docsExamined / nReturned proche de 1.



PyMongo : 1 MongoClient

Instance globale, pool interne, API identique à mongosh. Ne jamais recréer par requête.

→ **Demain — Jour 3** : la scalabilité horizontale — **Replica Sets, Sharding**, et leur couplage pour la haute disponibilité.